

Design of AXI4-Lite Crossbar Interconnect

Presented By:

Aman Chourasia

Omkar Bodake

Sai Desale

Arava Pavan Sri Sai

Aryan Rana

Project Group No. 3

Under the guidance of :

Ms. Keerthi & Mr. Praveen

Introduction

Modern SoC designs integrate processors, memory, and peripherals that must communicate efficiently. As system complexity grows, direct point-to-point connections become difficult to manage, requiring a structured interconnect architecture.

An AXI4-Lite Crossbar Interconnect connects multiple masters to multiple slaves using the AXI4-Lite protocol, handling address routing, arbitration, and read/write control while following AXI handshaking rules. This improves scalability, reliability, and overall system performance in embedded and FPGA-based designs.

Problem Statement

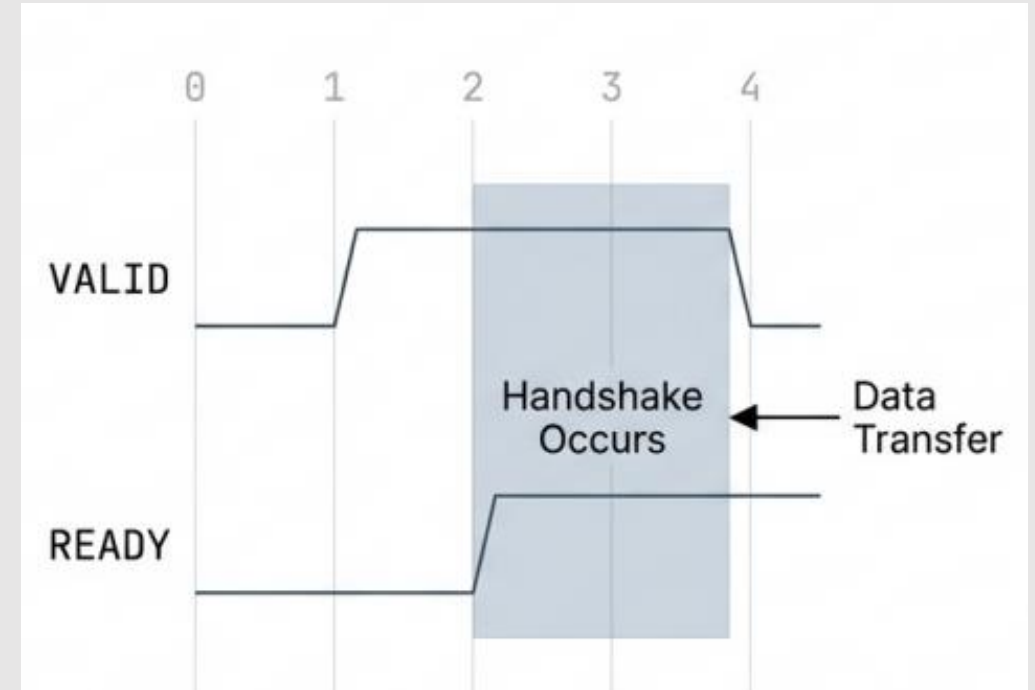
- Multiple AXI4-Lite masters need access to multiple slave peripherals
- Direct connections increase design complexity and wiring overhead
- Simultaneous master requests lead to contention and access conflicts
- Lack of proper arbitration can cause data inconsistency
- Existing simple interconnects are not scalable
- Need for a standardized AXI4-Lite based crossbar solution

Objectives

- Design and implementation of an AXI4-Lite Crossbar Interconnect enabling communication between multiple masters and multiple slaves.
- Address decoding mechanism that routes each transaction to the correct slave based on the address.
- Arbitration logic that resolves conflicts when multiple masters request access to the same slave.
- Control logic that manages AXI read and write channels while maintaining proper VALID–READY handshaking.
- Integration of skid buffers to handle stalls and prevent data loss during backpressure.

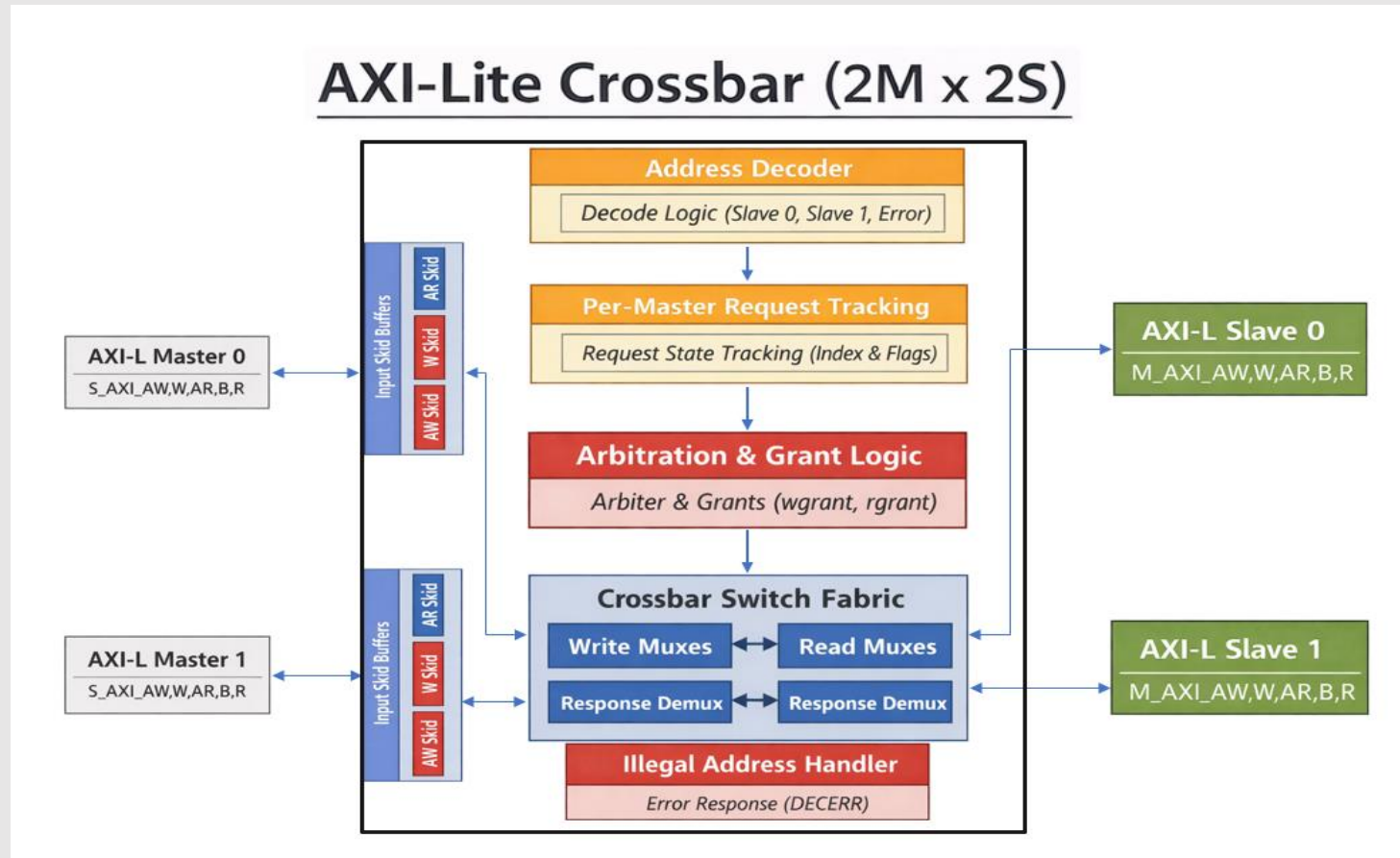
System Specifications

- Configuration: 2 Masters (NM=2), 2 Slaves (NS=2)
- Protocol: AX14-Lite (5 Channels)
- Throughput Goal: 1 Transaction / Clock Cycle
- Latency: Minimum 2 cycles (Arbitration + Routing)
- Parameters:
C_AXI_DATA_WIDTH=32,
C_AXI_ADDR_WIDTH=32



Block Design

AXI-Lite Crossbar Architecture:



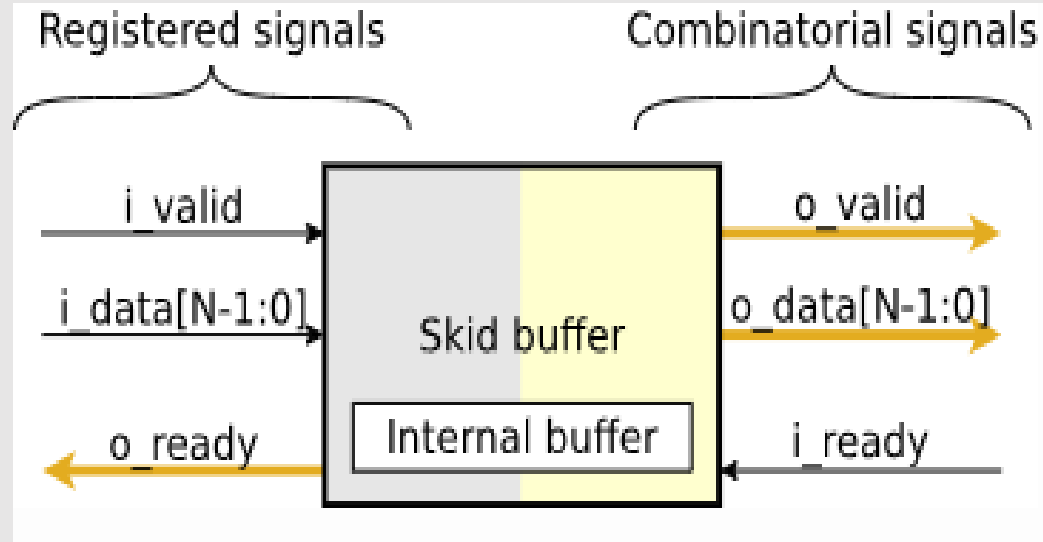
Skid Buffer

An input skid buffer is a small single-entry storage block placed between the AXI master and internal crossbar logic. It is used in READY–VALID handshake interfaces to temporarily hold one transaction when the downstream logic is not ready.

Purpose of Skid Buffer:

- To ensure safe data transfer in AXI VALID–READY handshake
- To prevent data loss during sudden stalls
- To break long combinational paths in control signals
- To improve timing performance and design stability

Skid Buffer Diagram



Signal	Meaning
<code>i_valid</code>	Input data is valid
<code>o_ready</code>	Buffer can accept data
<code>o_valid</code>	Output data is valid
<code>i_ready</code>	Next block ready
<code>i_data / o_data</code>	Actual address/data

Placed at each master input interface

- Used for AXI channels:
 - Write Address (AW)
 - Write Data (W)
 - Read Address (AR)
- Stores incoming transaction if arbitration or decoder delays READY signal.
Forwards stored data once downstream logic becomes ready

Working:

If Downstream READY = 1, Data passes directly

If Downstream READY = 0, Data stored in skid buffer

READY becomes 1 again, Stored data sent forward

Address Decoder

Address Decoder identifies which slave should handle an incoming AXI-Lite transaction. It examines the transaction address and maps it to a predefined slave address region

Why is it Required?

- Multiple slaves share the same interconnect
- Crossbar must route each request to only one valid slave
- Prevents incorrect or ambiguous access

Working of Address Decoder

Address Matching

- Each slave is assigned a base address and address range
- Incoming address is compared against all slave ranges

Decode Output

- Exactly one slave is selected for a valid address
- If address does not belong to any slave:
 - Transaction is marked as illegal

Pipelined Operation

- Decoder operates in a pipelined manner
- Uses flow control to pause decoding when downstream logic is busy
- Ensures stable address during routing decisions

Error Handling & Design Benefits

Illegal Address Handling

- Invalid addresses are detected early
- Crossbar generates a protocol-compliant error response
- No slave is accessed for illegal transactions

Design Advantages

- Modular and scalable for multiple masters and slaves
- Clean separation of responsibilities:
 - Address Decoder → *where to route*
 - Arbitration → *when to route*
- Improves reliability and correctness of the interconnect

Pending Request Tracking

What it does?

- Tracks ongoing read and write transactions for each master
- Knows whether a master is busy or idle

Why it is required?

- Enforces AXI-Lite transaction rules
- Prevents masters from over-issuing requests
- Ensures safe and correct response handling
- Supports stable arbitration and flow control

Pending Request Tracking

How this information is used

- Controls request acceptance
- Helps decide when a master can issue new requests
- Guides arbitration and channel release

How the Block Works (High-Level)

- Request accepted → pending count increases
- Response completed → pending count decreases
- Status flags update automatically
- Other crossbar blocks use this status to control flow

Signal / Register	Purpose
awpending / rpending	Count of active write / read transactions
swempty / srempty	Indicates no pending write / read requests
swfull / srfull	Indicates pending request limit reached

Arbitration

Arbitration is the mechanism that decides which master gets access to a slave when multiple masters request the same slave simultaneously. Since only one master can communicate with a slave at a time, arbitration prevents conflicts on the bus.

In an AXI-Lite crossbar system:

- Multiple masters share a limited number of slaves
- Requests can occur at the same time
- Only one connection per slave channel is allowed

Arbitration ensures:

- No data collision
- Fair channel usage
- Continuous data flow without blocking

Arbitration Implementation

Arbitration logic is implemented separately for write and read paths:

- Write Arbitration Block → ARBITRATE_WRITE_REQUESTS
- Read Arbitration Block → ARBITRATE_READ_REQUESTS

Grants are stored in signals like:

- wgrant[N] – Write grant for master N
- rgrant[N] – Read grant for master N
- mwgrant[M] – Indicates slave M is granted to a master (write)
- mrgrant[M] – Same for read

Arbitration Working

1. Masters generate request

Each master raises a request signal for a target slave.

```
wrequest[N][M] → Write request
rrequest[N][M] → Read request
```

2. Check if slave is available

Arbitration checks whether the slave is free or already granted.

3. Grant is issued

If available, grant is given:

```
wgrant[N] → Master N granted write channel
rgrant[N] → Master N granted read channel
```

4. Stay on channel

If transaction is still in progress, the grant is maintained.

5. Release channel

When no pending transactions exist, the channel is released.

Arbitration Signals

Signal	Purpose
wrequest / rrequest	Masters requesting slaves
wrequested / rrequested	Prevents multiple masters claiming same slave
wgrant / rgrant	Grant given to master
swgrant / srgrant	Master is actively connected
mwgrant / mrgrant	Slave is occupied

Linger Feature

The parameter OPT_LINGER introduces a delay before releasing a granted channel.

This avoids frequent switching and improves performance by keeping the connection alive for a few extra cycles after transaction completion.

Write Arbitration Flow

Request → Check slave availability → Grant → Data transfer → Transaction complete → Release

Read Arbitration Flow

Same process as write arbitration but for AR and R channels.

Control Logic

Control logic manages when transactions start, continue, and end inside the AXI-Lite Crossbar.

While arbitration decides who gets the channel, control logic decides how long the channel stays active and when to release it.

Control logic is responsible for:

- Managing AXI READY–VALID handshake
- Tracking pending transactions
- Ensuring write address, write data, and response occur correctly
- Handling read request and read response flow
- Preventing overflow using transaction counters
- Releasing bus grants at the correct time

Control Logic Functions

1. Channel Full / Empty Detection

swfull / srfull: Channel is full → block new requests

swempty / srempty: Channel empty → safe to release grant

2. Handshake Control

Control logic ensures proper AXI handshake:

VALID = request present

READY = receiver can accept

Transfer occurs only when $VALID \ \&\& \ READY = 1$

This avoids data corruption and protocol violation.

3. Grant Release Logic

Channel is released when: No pending transactions, linger timer expires (OPT_LINGER), another master requests the slave.

This prevents deadlocks and improves fairness

Control Logic Functions

4. Error Handling

- If address decoding fails, control logic:
- Routes request to error path
- Returns SLVERR response

- Write Channel Control Flow

AW accepted → W data accepted → Response generated → Counters updated → Channel released

- Read Channel Control Flow

AR accepted → Slave returns data → Response forwarded → Counters updated → Channel released

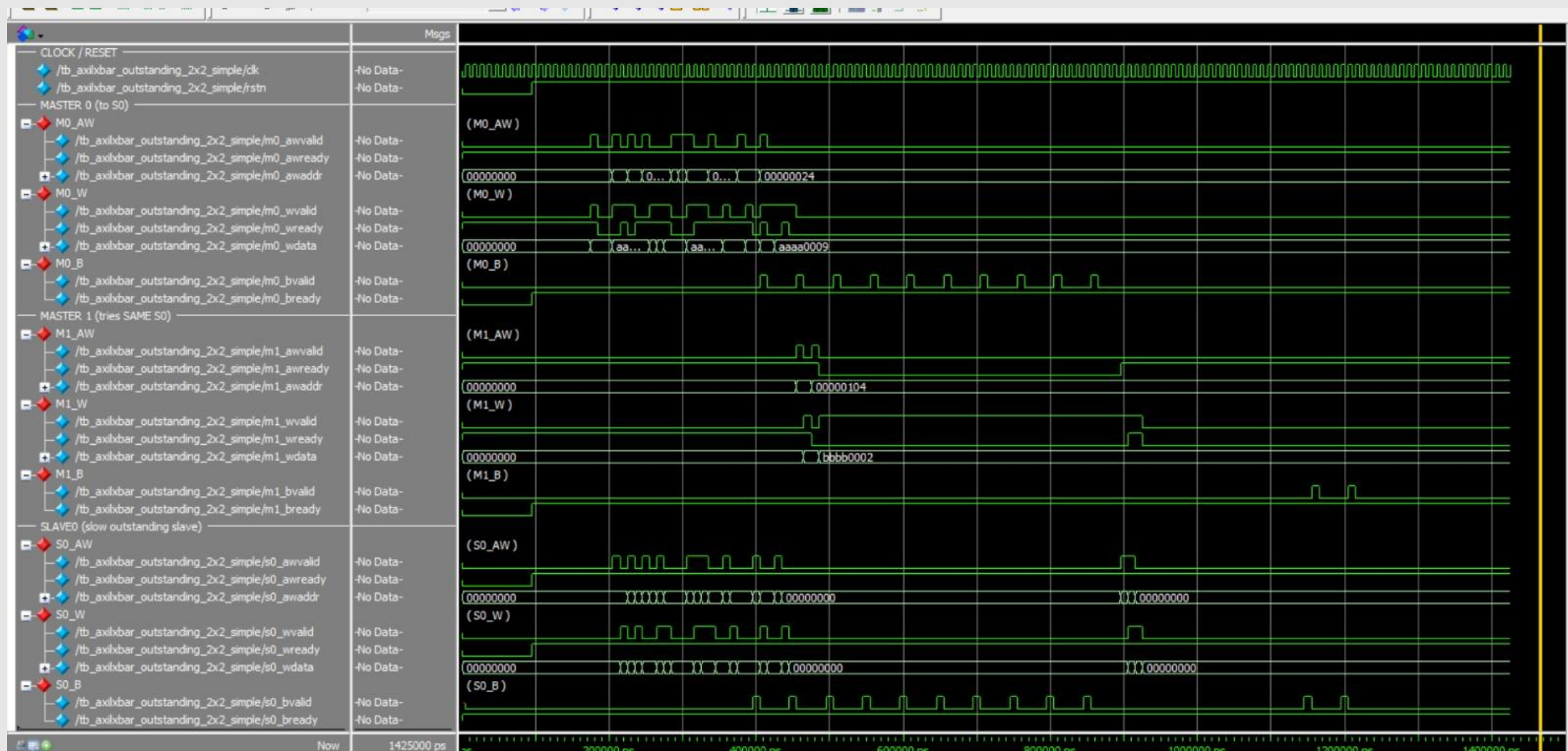
Error Handling

Sometimes a master may send an address that does not belong to any slave. The crossbar must handle this safely without breaking AXI protocol.

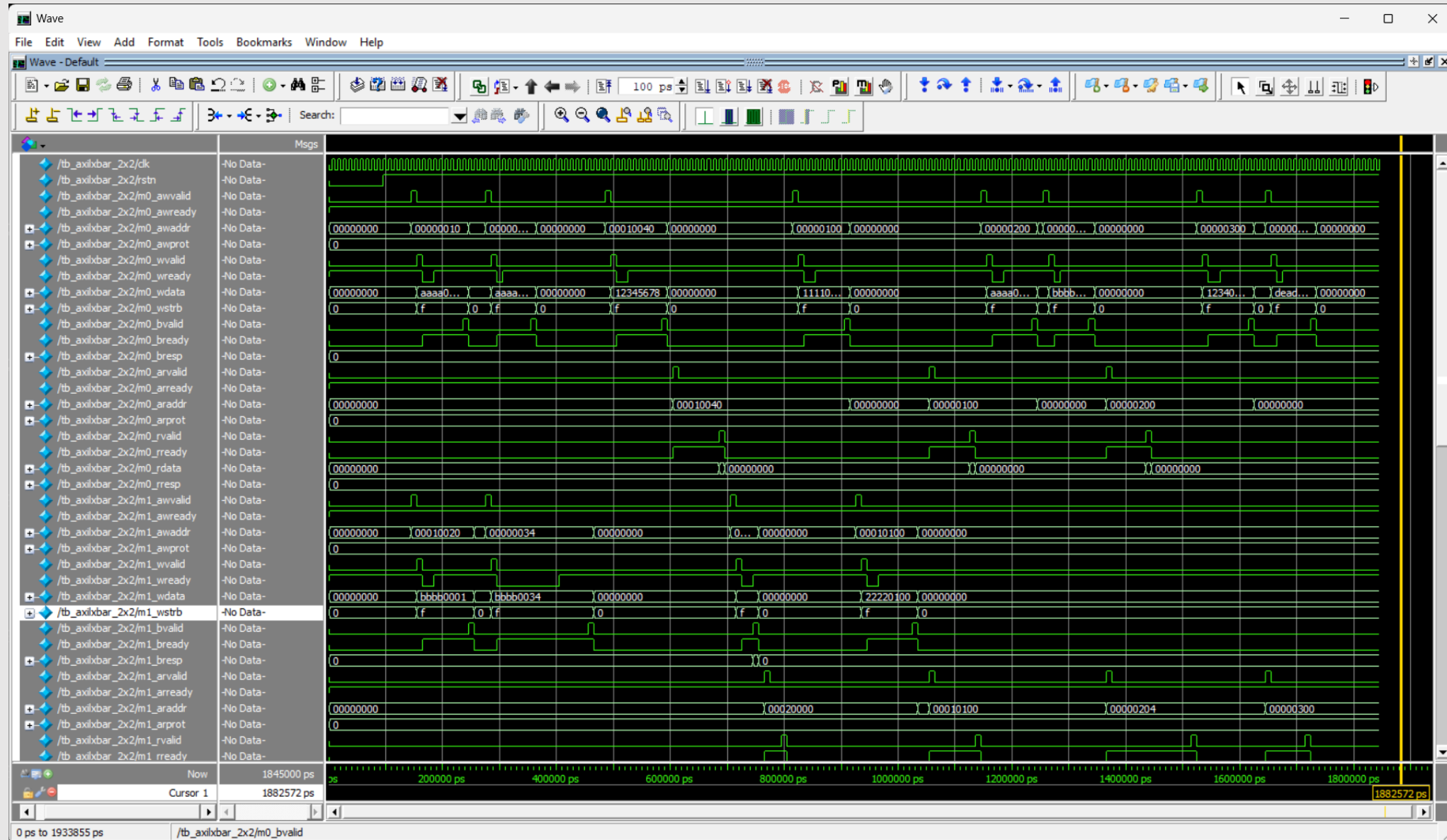
Error is detected as follows:

- Address Decoder checks:
 $(\text{address} \& \text{mask}) == (\text{slave_address} \& \text{mask})$
- If no match → Invalid address
- Request is routed to error channel
- No real slave is accessed
- Crossbar returns error response

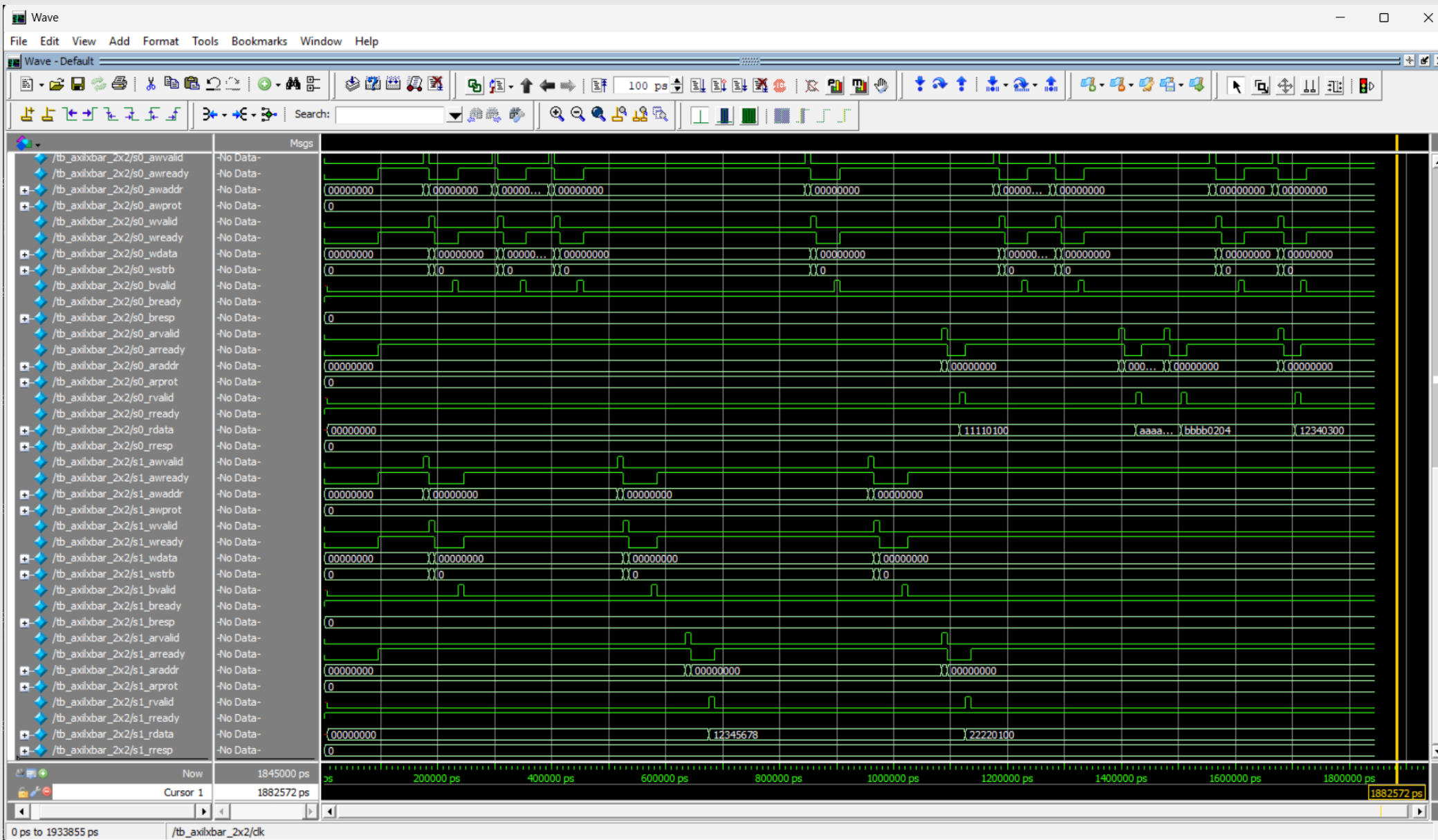
Simulation Results



Simulation Results



Simulation Results



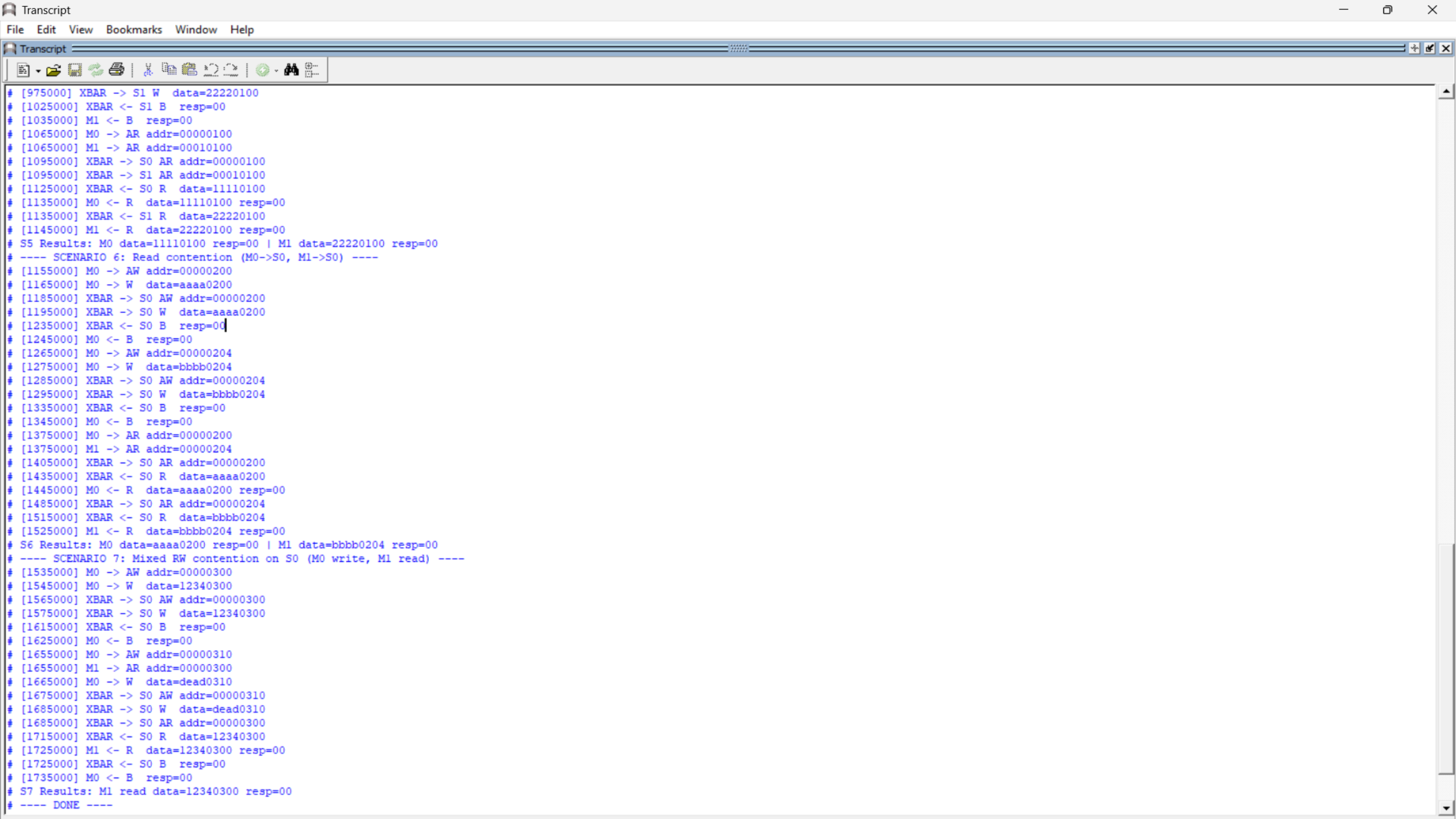
Transcript

File Edit View Bookmarks Window Help

Transcript

VSIM 9.7.2

----- SCENARIO 1: Parallel writes (M0->S0) and (M1->S1) -----
[155000] M0 -> AW addr=00000010
[155000] M1 -> AW addr=00010020
[165000] M0 -> W data=aaaa0001
[165000] M1 -> W data=bbbb0001
[185000] XBAR -> S0 AW addr=00000010
[185000] XBAR -> S1 AW addr=00010020
[195000] XBAR -> S0 W data=aaaa0001
[195000] XBAR -> S1 W data=bbbb0001
[235000] XBAR <- S0 B resp=00
[245000] M0 <- B resp=00
[245000] XBAR <- S1 B resp=00
[255000] M1 <- B resp=00
----- SCENARIO 2: Contention (both masters write S0) -----
[285000] M0 -> AW addr=00000030
[285000] M1 -> AW addr=00000034
[295000] M0 -> W data=aaaa0030
[295000] M1 -> W data=bbbb0034
[305000] XBAR -> S0 AW addr=00000030
[315000] XBAR -> S0 W data=aaaa0030
[355000] XBAR <- S0 B resp=00
[365000] M0 <- B resp=00
[405000] XBAR -> S0 AW addr=00000034
[415000] XBAR -> S0 W data=bbbb0034
[455000] XBAR <- S0 B resp=00
[465000] M1 <- B resp=00
----- SCENARIO 3: Read-after-write correctness -----
[495000] M0 -> AW addr=00010040
[505000] M0 -> W data=12345678
[525000] XBAR -> S1 AW addr=00010040
[535000] XBAR -> S1 W data=12345678
[585000] XBAR <- S1 B resp=00
[595000] M0 <- B resp=00
[615000] M0 -> AR addr=00010040
[645000] XBAR -> S1 AR addr=00010040
[685000] XBAR <- S1 R data=12345678
[695000] M0 <- R data=12345678 resp=00
----- SCENARIO 4: Illegal address (expect DECERR from xbar) -----
[715000] M1 -> AW addr=00020000
[725000] M1 -> W data=deadbeef
[755000] M1 <- B resp=11
[775000] M1 -> AR addr=00020000
[805000] M1 <- R data=00000000 resp=11
----- SCENARIO 5: Parallel reads (M0->S0) and (M1->S1) -----
[825000] M0 -> AW addr=00000100
[835000] M0 -> W data=11110100
[855000] XBAR -> S0 AW addr=00000100
[865000] XBAR -> S0 W data=11110100
[905000] XBAR <- S0 B resp=00
[915000] M0 <- B resp=00
[935000] M1 -> AW addr=00010100
[945000] M1 -> W data=22220100
[965000] XBAR -> S1 AW addr=00010100



Applications

1. AXI4-Lite Crossbar Interconnect is widely used for control and configuration communication in modern SoC and FPGA systems. It is designed for low-speed register access rather than high data transfer, making it ideal for managing peripherals and hardware modules.
2. In SoC designs, AXI4-Lite connects processors like ARM Cortex or RISC-V to control peripherals such as GPIO, UART, SPI, I²C, and timers. It allows the processor to write configuration values and read status registers of these devices.
3. In industrial automation systems, AXI-Lite is used in PLCs and controllers to manage ADC/DAC modules, motor drivers, and monitoring hardware. It ensures smooth control of industrial processes.

Conclusion

In this project, we designed a parameterized AXI-Lite crossbar to enable efficient communication between multiple masters and slaves. The design performs address decoding, arbitration, and transaction routing while complying with AXI-Lite protocol requirements. It supports backpressure handling, detects illegal addresses, and provides error responses, ensuring reliable operation. The modular and scalable architecture allows easy integration into larger systems and strengthens understanding of interconnect and protocol-based hardware design.

Thank you!